Airline Reservation System Video Guide

Prepared for JahMari Harrigan. Use this guide to explain the actual implementation in your own words. Record yourself and the running application. This guide supports the assignment’s required sequence; it is not a video or evidence that a presentation has been recorded.

Before recording

Finish the three user sessions and final browser checks. Start MySQL and the Node.js application, use fictional demo accounts, and verify the customer and administrator flows. Open the reports, schema, SQL analysis, test evidence and Git history. Increase editor/browser text size for readability. Hide .env, passwords, tokens and unrelated browser tabs. Do not display credentials in the recording.

Segment	Suggested duration
Introduction and scope	1 minute
Problem analysis and database design	2–3 minutes
Interface and workflow	1–2 minutes
Implementation and version history	2 minutes
Live customer and administrator demo	3–4 minutes
Testing, user feedback and reflection	2–3 minutes

Opening you can adapt

“I’m JahMari Harrigan. Airline Reservation System is a classroom reservation system using Node.js, HTML, CSS and MySQL. It supports one-way bookings for up to four passengers and includes an analytics dashboard built from synthetic data. I used Codex assistance and recorded the changes and tests. I’ll explain the design choices and demonstrate the implemented workflows.”

Explain the Analysis and Design

Problem definition

Explain who needs the system and why inconsistent reservation records are a problem. State the single-flight scope, fictional data and simulated payments. Point to measurable criteria: correct fares, saved records, no duplicate active seats, ownership checks and consistent cancellation.

Database model

Show the ER diagram and distinguish Flight from FlightInstance. Trace User -> Booking -> BookingPassenger -> Passenger. Trace SeatAssignment to Seat and FlightInstance. Explain why aircraft seat labels alone are not enough: seat 1A can exist on multiple aircraft and many departures. Explain how the composite foreign keys preserve that relationship.

Normalization

Start with a row containing passenger-name and seat lists. Explain atomic values in 1NF, removal of partial dependencies in 2NF, and separation of airport/aircraft/customer descriptions in 3NF. Give an update anomaly example. Explain that the saved booking fare is a historical snapshot, not a mistaken copy of the current selling fare.

Keys and integrity

Point to primary keys, foreign keys, alternate keys and secondary indexes. Explain that an index helps lookup performance while a foreign key enforces a relationship. Show UNIQUE(instance_id, active_seat). Explain that released allocations have NULL active_seat, allowing historical records while preventing two active allocations.

Demonstrate the Running Application

Action	What to explain
Register or log in	Fictional customer identity; public registration cannot create admins
Search a future route/date	UTC labels, empty states, dated departures
Open flight details	Fare, schedule, aircraft and available capacity
Book two passengers	Different seats; server calculates total; simulated payment
Show confirmation	Unique reference, saved fare and passenger seats
Retrieve in My bookings	Only owned records are returned
Cancel and revisit availability	Status changes; seats released; payment voided
Log in as administrator	Role separation is enforced on server
Create or edit a departure	Fare change affects new bookings; cancellation affects all related bookings
Open analytics and CSV	Explain metric definitions and synthetic provenance

If anything fails, pause and fix it before recording the final take. Do not describe a wireframe as a running screen. Do not claim real-world revenue or customer demand from the synthetic dataset.

Explain Implementation and Analytics

Version history

Show alpha, beta and production tags and their real commits. Alpha established the transactional customer flow. Beta added administration, fare preservation and route analysis. The final refinement added reproducible analytics data, throttling/origin checks, full-capacity tests and documentation. Show one concrete before/after example and the related test.

Booking transaction walkthrough

Explain validation, authentication, the locked FlightInstance row, server-derived price, inserted booking/passengers/assignments/payment, and commit. Explain what happens when the second passenger’s seat is already occupied: an error causes rollback, so the first passenger’s insert does not survive. Show T06 and T07 evidence.

Data analyst portfolio explanation

Show analytics/analysis.sql. Explain grain before joins, why capacity must be aggregated once per departure, how cancellation rate is calculated, and why net booked value is not recognized revenue or profit. Show the CTE and DENSE_RANK query. Use the integrity exception report before interpreting results.

The reproducible scenario contains 660 reservations, 528 confirmed and 132 cancelled, giving a designed 20% cancellation rate. Net booked value is $105,080. June and August have $24,620 each; July and September have $27,920 each. These differences come from the generation rules and route/fare mix, not observed seasonality or causal business effects.

Present Evaluation and Reflection

Testing evidence

Show the actual production and clean-setup logs and the server-restart check. Explain expected versus observed outcomes for concurrency, rollback, cancellation, full capacity and unauthorized access. State which checks were automated and which were performed manually.

Three real participants

Only after sessions are completed, show participant codes, dates, tested versions/devices, task completion and actual ratings. Quote actual comments. Explain each change made in response and whether it was retested. If sessions are still pending, the evaluation requirement is not complete; do not fill it with invented results.

Personal reflection prompts

What database design decision can you now explain better? What error did you investigate yourself? How did you verify an AI-generated change? What did a real user notice that a test missed? What would you improve next? Answer from your own experience rather than reading claims about learning you have not yet done.

Likely instructor questions

Why use a junction table? What prevents a race condition? Why store fare snapshots? What is the difference between FK and secondary index? What happens if the server stops? Can one customer cancel another’s booking? How do you know the analytics do not double-count revenue? What is simulated and what is persistent? Practice answering with the corresponding source file open.

Recording and Submission Checklist

Record clear narration and readable screen text. Follow the rubric’s seven-section sequence. Include a real booking, confirmation, cancellation and administrator operation. Show database or API evidence supporting persistence and seat release. Keep the final video public or unlisted as the assignment permits, and verify the instructor can open it.

Item	Completion action
GitHub	Keep repository private; invite the instructor separately if required
Links.txt	Add the actual repository and YouTube URLs after publication
Reports	Confirm one-page Problem Definition and five-page Problem Analysis in final Word rendering
Evaluation	Attach actual three-user results, fixes and retests
ZIP	Include source, lockfile, schema, seeds, reports, logs and evidence; exclude secrets and node_modules
Resume	Deferred at your request; assignment still lists it as a submission requirement
CLI evidence	Add a real Codex CLI session if the instructor requires CLI rather than desktop Codex
Final review	Explain each major file and query before presenting it as portfolio work

Do not say the project is fully submitted until the missing human evaluations, browser evidence, GitHub access and your video are complete. The source and guide can support that work, but cannot substitute for it.